An Activity-Rate-Aware Hysteresis Controller for Lightweight Adaptive Power Gating in SoC Functional Blocks

Authors: Joud Thaher & Layan Salem
Supervisor: Dr. Khader Mohammed
Department: Electrical and Computer Engineering
University: Birzeit University, Palestine
GitHub Repository: https://github.com/JHT127/hysteresis-adaptive-power-gating

***Abstract*—Conventional power-gating controllers for small SoC functional blocks typically select sleep states using idle-cycle count alone, a policy that is vulnerable to thrashing – repeated, costly sleep/wake transitions – under bursty workloads. This work proposes a lightweight power-gating controller for a 32-bit ALU block that gates sleep-state entry on the conjunction of consecutive idle-cycle count and a rolling-window activity rate, with adaptive widening of entry thresholds during high-activity periods. The controller is implemented in synthesizable RTL with three power states (ON, Light Sleep, Deep Sleep) and is evaluated against a fixed-threshold baseline representative of prior multi-level power-gating work, using an identical ALU, isolation strategy, and UPF power intent for both designs. Functional simulation across burst, sparse, and mixed workloads shows the proposed controller eliminates thrashing under bursty traffic (zero mode transitions vs. nineteen for the baseline, preserving throughput at 59.3% vs. 19.3%), while converging to comparable leakage savings under unambiguous idle conditions (31.6% vs. 33.4%). Synthesis at two clock corners (5 ns and 2 ns) and full place-and-route in a 14 nm standard-cell library using Synopsys Fusion Compiler show a real post-route area overhead of 13.6% with identical, fully met timing closure on both designs and zero open nets after routing. Results indicate that the proposed dual-condition policy trades a bounded, modest area cost for a substantial improvement in throughput stability under realistic bursty traffic – a trade-off not previously quantified for comparator-based (non-machine-learning) adaptive power-gating controllers at this scale.**

***Index Terms*—power gating, adaptive power management, leakage power, low-power VLSI, hysteresis control, multi-level sleep, RTL design, ASIC implementation, SAED 14nm**

I. Introduction

Leakage power has become a dominant component of total power dissipation as CMOS technology scales into deep-submicron and nanometer nodes, particularly for blocks that spend significant fractions of their operating lifetime idle [1], [2]. Power gating – disconnecting idle logic from its supply rail via sleep transistors – remains one of the most widely deployed leakage-reduction techniques in commercial SoC design, but the *policy* governing when to gate is frequently static: a fixed number of idle cycles triggers sleep entry, regardless of how that idle period arose [3].

This static-threshold approach has a known failure mode. A workload composed of short, frequent bursts separated by gaps that individually exceed the sleep-entry threshold will cause the controller to repeatedly enter and exit sleep states – thrashing – even though the block is, in aggregate, far from idle. Each entry/exit cycle incurs a wake-up latency penalty and, in deep-submicron designs, a rush-current and ground-bounce cost during reactivation [4]. The leakage saved by sleeping briefly can be outweighed by the throughput and energy cost of waking up repeatedly.

Recent work has approached this problem from two directions. Multi-level sleep-mode controllers [3] introduce intermediate power states with different leakage/wake-up-latency trade-offs, but still gate transitions on idle-cycle count alone. Workload-adaptive controllers using machine-learning techniques – clustering and recurrent prediction – have been proposed for multi-core systems [5], achieving strong adaptivity at the cost of prediction-hardware overhead that is difficult to justify for a single small functional block.

This work proposes a middle path: a comparator-based (non-machine-learning) hysteresis controller that gates sleep-state entry on the *conjunction* of idle-cycle count and a rolling-window activity rate, widening its entry thresholds when recent activity indicates a bursty rather than genuinely idle workload. This work makes the following contributions:

- A dual-condition sleep-entry policy combining idle-cycle count and activity rate, implemented with a 16-cycle shift-register popcount and simple comparators – no multipliers, no learned models.
- Adaptive threshold widening that suppresses sleep entry during bursty operation without per-cycle software classification.
- A fair, single-module-substitution comparison methodology against a fixed-threshold baseline sharing identical ALU, isolation, retention, and UPF infrastructure, isolating the decision-logic contribution as the sole experimental variable.
- A quantitative characterization, at both RTL-simulation and post-route physical-implementation levels, of the throughput-stability/area trade-off this policy introduces.

The remainder of this paper is organized as follows. Section II reviews related power-gating and adaptive power-management work. Section III describes the proposed architecture. Section IV details the implementation and experimental

methodology. Section V presents results. Section VI discusses findings and limitations. Section VII concludes.

II. Background and Related Work

Agarwal et al. [3] established the multi-level sleep-mode concept this work builds on, defining sleep depths with different footer gate-bias levels and demonstrating that allowing an intermediate, state-retentive mode in addition to full cutoff yields a further 17% leakage reduction over single-mode gating, with the retentive mode itself saving 19% leakage while preserving circuit state. Their decision policy, however, selects mode purely from idle-cycle count against fixed thresholds; the present work adopts their mode-hierarchy concept directly but replaces the decision logic.

Chang et al. [4] address a complementary problem: once a sleep-mode transition has been decided, how to execute the wake-up safely. Their two-phase footer activation – enabling a small transistor fraction before the full width – reduces wake-up time by approximately 10% while bounding power-supply bounce. This work's sleep controller adopts the same two-phase activation principle for both the baseline and proposed designs, so it is held constant across the comparison reported here; the contribution of this work is orthogonal, addressing the entry-decision policy rather than the transition mechanism.

Karthikeyan et al. [5] propose workload-adaptive power management for multi-core systems combining real-time monitoring, dynamic voltage and frequency scaling, and core-level gating, using K-means clustering and recurrent neural prediction to classify workload intensity. Reported savings of up to 35% and meaningful thermal reduction demonstrate the value of activity-aware adaptation, but the prediction hardware is sized for multi-core systems and is difficult to justify for a single small block such as the one considered here. This work targets the same underlying goal – workload-character-aware adaptation – using a fixed-function comparator network rather than a learned model.

Surveys by Shiva and Patil [1] and Sahu et al. [2] situate power gating within the broader landscape of low-power VLSI techniques (dynamic voltage scaling, clock gating, multi-threshold CMOS), both noting that AI-assisted predictive management is an emerging direction but that lightweight, non-learned alternatives remain comparatively underexplored for fine-grained blocks. Tong et al. [6] address adaptive energy management in an unrelated domain (mobile edge computing task offloading), offering a Lyapunov-based stability framework for resource allocation under stochastic load; while not directly applicable to gate-level power gating, the queue-stability framing motivates this work's attention to throughput preservation, not leakage savings alone, as an evaluation criterion.

Table I summarizes the approach, key strength, and remaining gap of each reviewed work relative to the present contribution.

No reviewed work combines a non-learned, comparator-based dual-condition (idle-count and activity-rate) sleep-entry policy with adaptive threshold widening, evaluated against a fair, structurally matched fixed-threshold baseline at both RTL-simulation and post-route physical-implementation levels. This is the contribution of the present work.

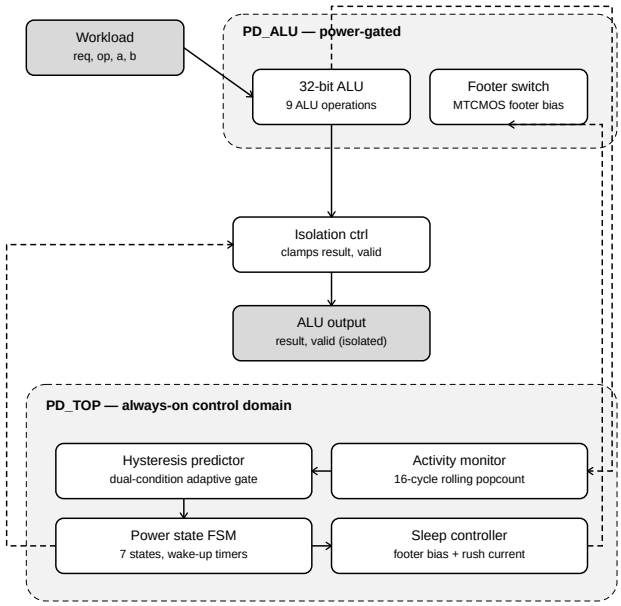

Fig. 1. Architecture of the adaptive multi-level power-gating controller, showing the two UPF power domains (PD_ALU, power-gated; PD_TOP, always-on) and inter-module signal flow.

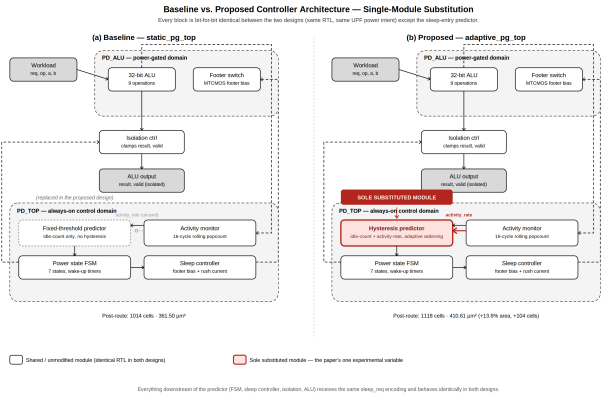

Fig. 2. Baseline vs. proposed controller architecture under the single-module substitution protocol. Every block is bit-for-bit identical between the two designs except the sleep-entry predictor (highlighted in red). Post-route instance counts and area are shown for reference.

III. Proposed Architecture

The controller is organized into two power domains (Fig. 1), following the UPF power-intent model. PD_TOP, the always-on domain, contains the activity monitor, hysteresis predictor, power-state finite state machine (FSM), and sleep controller. PD_ALU, the power-gated domain, contains the 32-bit ALU functional block and its MTCMOS footer switch. Fig. 2 shows both the baseline and proposed top-level architectures side by side, highlighting the single substituted module.

TABLE I
COMPARISON OF RELATED WORK IN POWER GATING AND ADAPTIVE POWER MANAGEMENT

Reference	Approach	Key Strength	Key Limitation	Gap Addressed by This Work
Agarwal et al. [3]	MTCMOS multi-level power gating (Active/Light Sleep/Deep Sleep) with fixed idle-cycle thresholds	Proven leakage savings; retentive intermediate mode cuts leakage by a further 17%	Static thresholds; no activity-rate check, prone to thrashing under bursty idle gaps	Adopts the mode hierarchy directly but replaces the fixed-threshold decision logic with a dual-condition, activity-rate-aware gate
Chang et al. [4]	Two-phase footer activation for rush-current-controlled wake-up	Reduces wake-up time by ∼10% while bounding supply bounce	Addresses only the wake-up transition mechanism; no sleep-entry decision logic	Adopts the wake-up mechanism unchanged in both baseline and proposed designs; contribution is orthogonal (entry policy, not transition)
Karthikeyan et al. [5]	ML-based workload monitoring, DVFS, and core-level power gating for multi-core SoCs	Full-stack adaptive power management; reported savings up to 35%	Clustering/recurrent-prediction hardware overhead is difficult to justify for a single small block	Achieves comparable qualitative adaptivity with a fixed-function comparator network – no learned model, no inference hardware
Shiva and Patil [1]	Survey of DVS, power gating, and clock-gating optimization techniques	Comprehensive overview of the low-power VLSI landscape	No new architecture or hardware implementation proposed	Identifies lightweight, non-learned adaptive gating as underexplored, motivating this work's scope
Sahu et al. [2]	Design-optimization survey for low-power VLSI circuits in high-performance computing	Broad, practically grounded design-optimization perspective	No adaptive, rate-aware sleep-entry logic proposed	Reinforces the same gap; this work supplies a concrete circuit-level instantiation
Tong et al. [6]	Lyapunov-based stability framework for resource allocation in IIoT task offloading	Principled queue-stability framing for throughput-aware adaptation	Different domain (network resource allocation); not applicable at gate level	Motivates evaluating throughput preservation, not leakage savings alone, as a first-class metric

A. Activity Monitor

The activity monitor implements a 16-cycle rolling window as a shift register, computing a consecutive-idle-cycle count and an activity rate (fraction of the window with observed activity, scaled to an 8-bit range) using only shift and population-count logic – no multipliers. Every cycle, a 1 is shifted in if the ALU was active, or a 0 if idle. The population count of the window gives the activity rate.

B. Hysteresis Predictor

The hysteresis predictor is the controller's central novel contribution. It requests a sleep-state transition only when *both* the idle-cycle count exceeds a mode-specific threshold *and* the activity rate is below a corresponding ceiling; further, when the activity rate exceeds a configurable high-activity threshold (`HIGH_RATE_THRESH` $= 192/255$), both entry thresholds are widened by a fixed offset (`ADAPT_OFFSET` $= 6$ cycles), suppressing sleep entry during workloads that are bursty rather than genuinely idle. Separate, narrower exit thresholds create hysteresis bands that prevent rapid oscillation between states.

The dual-condition sleep-entry gate operates as follows:

$$\text{enter\_LS} = (\texttt{idle\_count} > \theta_{\text{LS}}) \wedge (\texttt{rate} < R_{\text{LS}}) \quad (1)$$

$$\text{enter\_DS} = (\texttt{idle\_count} > \theta_{\text{DS}}) \wedge (\texttt{rate} < R_{\text{DS}}) \quad (2)$$

where θ_{LS} and θ_{DS} are the adaptive thresholds:

$$\theta_i = \begin{cases} \theta_{i,\text{base}} + \Delta & \text{if } \texttt{rate} > T_{\text{high}} \\ \theta_{i,\text{base}} & \text{otherwise} \end{cases} \quad (3)$$

with $\theta_{\text{LS,base}} = 4$, $\theta_{\text{DS,base}} = 8$, $\Delta = 6$, $R_{\text{LS}} = 80$, $R_{\text{DS}} = 32$, and $T_{\text{high}} = 192$ (all in 8-bit counts or fractions of 255).

Figs. 3 and 4 show the synthesized gate-level schematics of the baseline and proposed predictors respectively, extracted from the DC Ultra netlist. The baseline predictor (Fig. 3) uses only `idle_count[7:0]` as input, implementing threshold comparison through an OR4NR2INVAO32 chain with two output flip-flops (FDP, FSDPQB) – a total of approximately 6 standard cells. The proposed predictor (Fig. 4) adds `activity_rate[7:0]` as a second input port, requiring additional NR2, AOI21, OAI21, and OR4 cells to implement the conjunction of both conditions and the hysteresis state – approximately 16 standard cells. The 104-cell post-route overhead between the two designs includes this predictor difference plus the additional comparator and MUX logic implementing the adaptive threshold widening (Eq. 3).

C. Power-State FSM

The power-state FSM (Fig. 5) mediates between the predictor's requested mode and the physically realized power state, inserting fixed wake-up-latency delays (3 cycles from Light Sleep, 8 cycles from Deep Sleep) and asserting isolation one cycle before the sleep signal to the power domain, ensuring no floating or undefined values propagate from the gated domain during a transition. The FSM has seven states: `S_ON`, `S_TO_LS`, `S_LIGHT_SLEEP`, `S_TO_DS`, `S_DEEP_SLEEP`, `S_WAKEUP_LS`, and `S_WAKEUP_DS`. Leakage in Light Sleep is approximately 81% of ON-state leakage; Deep Sleep reduces leakage to approximately 64% of ON-state leakage.

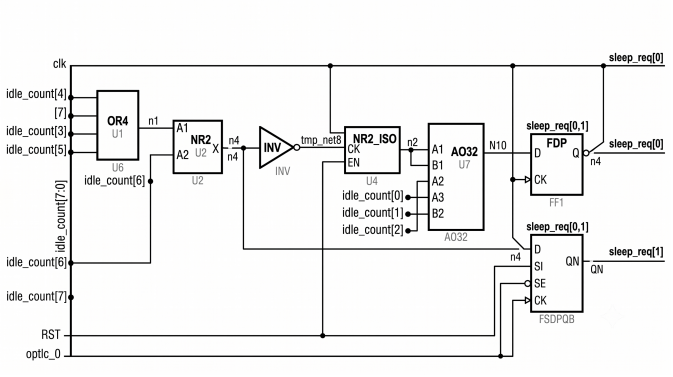

Fig. 3. Synthesized gate-level schematic of the *baseline* fixed-threshold predictor (DC Ultra, SAED 14 nm LVT). Only `idle_count[7:0]` is consumed; `activity_rate` is not connected. The decision logic reduces to a single OR4NR2INVAO32 comparator chain followed by two output registers (≈6 standard cells).

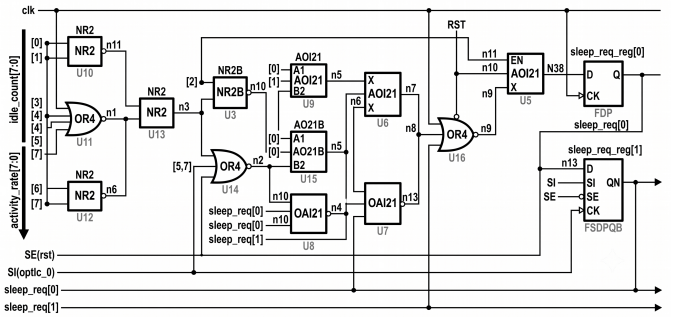

Fig. 4. Synthesized gate-level schematic of the *proposed* hysteresis predictor (DC Ultra, SAED 14 nm LVT). Both `idle_count[7:0]` and `activity_rate[7:0]` are consumed. The additional NR2, AOI21, OAI21, and OR4 cells implement the dual-condition AND gate and hysteresis state logic (≈16 standard cells), with output registered in FDP and FSDPQB flip-flops.

D. Sleep Controller and Isolation

The sleep controller sequences the footer transistor's gate bias across four levels (active, two intermediate biases, full cutoff) and implements two-phase rush-current control during wake-up, following [4]. The isolation controller clamps the ALU's output and valid signal to a safe constant whenever the domain is not fully powered, modeled as the behavioral equivalent of standard-cell ISO_AND isolation cells inserted automatically by the physical-design tool under UPF control.

IV. Implementation and Methodology

A. Baseline Design

To isolate the hysteresis predictor's contribution as the sole experimental variable, a baseline controller was constructed via single-module substitution: the hysteresis predictor is replaced with a fixed-threshold predictor implementing Agarwal et al.'s idle-cycle-count-only policy [3] – no activity-rate input, no hysteresis band (entry and exit thresholds equal), no adaptive widening. Every other module – the ALU, activity monitor, power-state FSM, sleep controller, isolation controller, and

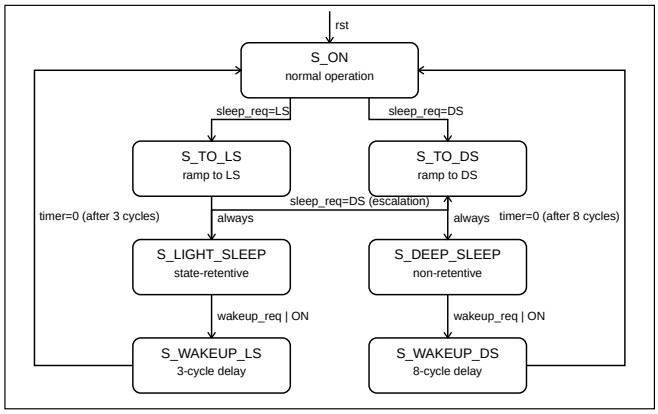

Fig. 5. Power-state FSM, showing all seven states and the conditions governing transitions between them. Self-transitions are omitted for clarity.

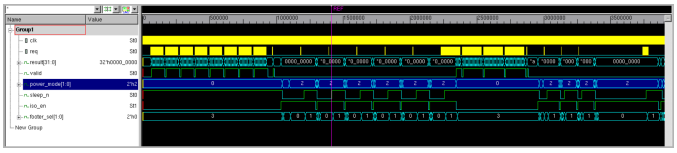

Fig. 6. VCS simulation waveform (SPARSE workload, time 1.41.9 ms). The controller enters Deep Sleep (`power_mode`=2, `sleep_n`=0, `iso_en`=1) during long idle periods and executes the 8-cycle wake-up sequence (`power_mode`: 2→0→1) when `req` is asserted. Isolation deasserts exactly one cycle after `sleep_n` rises, confirming correct sequencing.

UPF power intent – is shared, unmodified, between baseline and proposed designs (Fig. 2). Threshold values are matched: $\theta_{\text{LS,base}} = 3$ and $\theta_{\text{DS,base}} = 8$ for the baseline predictor, equal to the proposed design's base thresholds before any widening.

B. RTL and Verification

Both designs are implemented in synthesizable Verilog and verified in Synopsys VCS (T-2022.06-SP2) against a shared, parameterized testbench applying identical stimulus to both designs under burst, sparse, and mixed workload patterns. A correctness checker confirms zero output mismatches between designs whenever both report valid results, confirming the substitution introduces no functional regression in the shared datapath. VCD waveform files were generated for validation; Fig. 6 shows a representative waveform excerpt from the SPARSE workload.

C. Synthesis and Physical Implementation

Both designs were synthesized in Synopsys Design Compiler Ultra (R-2020.09-SP5) against a 14 nm low-voltage-threshold standard-cell library (SAED14LVT, `saed14lvt_tt0p8v25c`) at two clock corners: low-power (LP) at 5 ns (200 MHz) and high-speed (HS) at 2 ns (500 MHz), with clock-gating insertion enabled and leakage optimization active. Both designs were subsequently placed, clock-tree-synthesized, and routed in Synopsys Fusion Compiler (U-2022.12-SP5) at the 5 ns corner. The floorplan

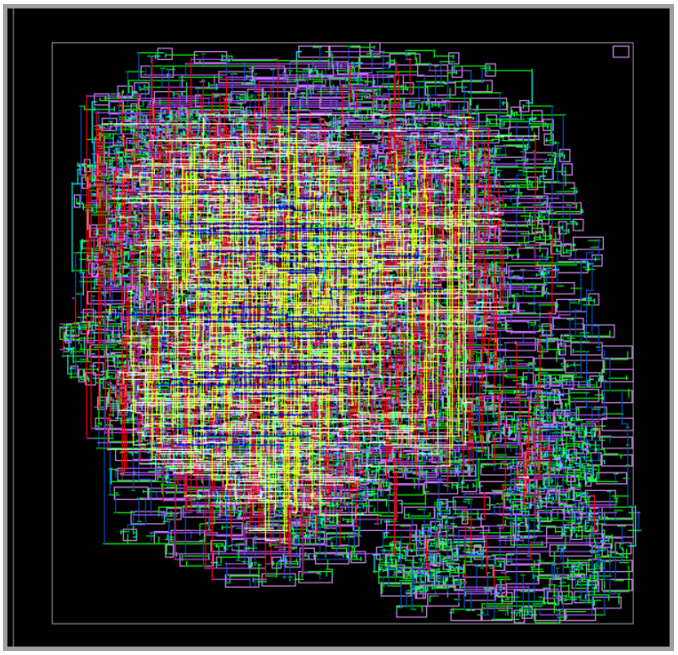

Fig. 7. Post-route layout of the proposed `adaptive_pg_top` design in SAED 14 nm LVT (Synopsys Fusion Compiler U-2022.12-SP5). Die area: $30\times30\,\mu\text{m}^2$, core utilization: 45.83%, 1,118 standard-cell instances, 9,873 μm total wire length, 0 open nets. Metal routing layers M1M9 are visible.

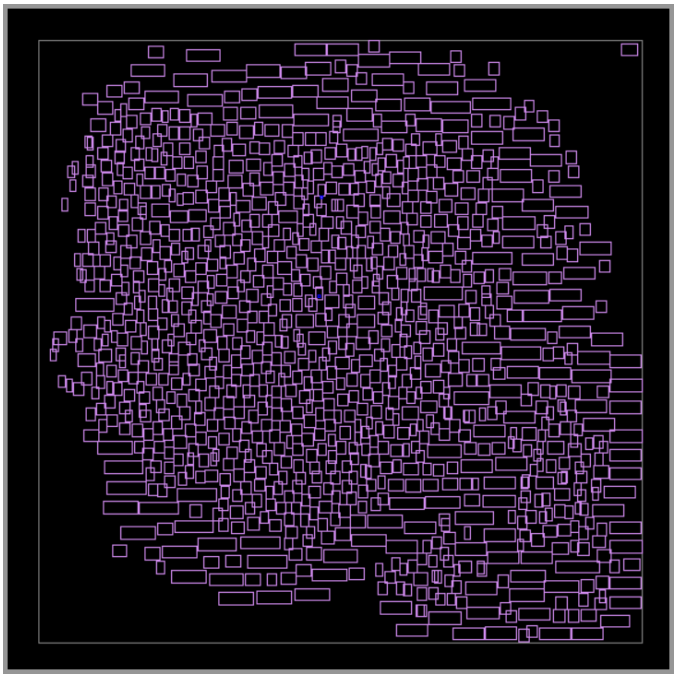

Fig. 8. Cell placement view with all routing layers hidden, showing the 1,118 SAED 14 nm LVT standard-cell instances distributed across the $30\times30\,\mu\text{m}^2$ core area at 45.83% utilization.

was initialized at $30\times30\,\mu\text{m}^2$ with 2 μm core offset, achieving 45.83% core utilization. Power and ground connections were established for 2,287 pins. Placement completed with zero violations; routing completed with zero open nets and 9,873 μm total wire length. Post-route worst negative slack is 3.25 ns (MET). The post-route layout is shown in Fig. 7.

D. Experimental Design

Three synthetic workloads – burst (frequent short bursts with idle gaps exceeding the baseline's fixed threshold but maintaining duty cycle above the proposed design's activity-rate widening trigger), sparse (long, unambiguous idle periods), and mixed (alternating dense and sparse phases) – were applied to both designs with identical pseudo-random operand stimulus, with per-mode cycle counts, transition counts, wake-up events, and valid-result throughput logged for each.

V. Results

A. Functional Comparison

Table II reports per-mode cycle distribution across the three workloads; Table III reports transition counts, throughput, and leakage savings; Fig. 9 visualizes the key results.

Under bursty traffic, the baseline controller's idle-cycle-only policy enters Light Sleep on every gap exceeding its fixed 3-cycle threshold, producing 19 mode transitions and 9 full wake-up events across the workload. Each transition incurs up to 3 cycles of wake-up latency ($19\times3=57$ cycle-equivalents of overhead), reducing valid-result throughput to 19.3%. The proposed controller's activity-rate gate, widened by the workload's sustained high duty cycle, suppresses sleep entry entirely (zero transitions), preserving throughput at 59.3% – a 3.1$\times$ improvement. Under sparse traffic, where idle periods are long and unambiguous, both controllers correctly progress to Deep Sleep with comparable leakage savings (33.4% baseline, 31.6% proposed), confirming the proposed design's added caution does not cost savings when idle conditions are genuine. Zero output mismatches were recorded across all workloads, confirming functional correctness of the single-module substitution.

B. Physical Implementation Results

Tables IV VII present the synthesis and post-route PPA results decomposed by metric category; Fig. 10 visualizes the area trade-off.

The area results (Table IV) show a 13.6% real post-route overhead, materially lower than the 20.4–26.1% suggested by pre-layout DC estimation, reflecting tighter placement achieved by the Fusion Compiler router versus the wireload interconnect model. At the HS (2 ns) corner, area overhead increases from 20.4% to 26.1% as DC inserts stronger drive-strength cells; both corners achieve positive worst-case slack (Table V), confirming timing feasibility across the explored frequency range. Post-route timing improves relative to DC estimates (WNS +3.25 ns post-route vs. +2.51 ns at DC synthesis) due to the router achieving shorter interconnect paths than the wireload model predicts.

The dynamic power overhead (Table VI) reflects the additional combinational logic in the always-on PD_TOP domain: cell internal power increases by 91.0% and switching power by 19.7%, for a total dynamic overhead of 62.5%. This is the at-speed cost of the controller logic itself. The post-route static leakage overhead of 20.2% (Table VII) similarly measures the always-on controller cost at full supply; the realized runtime

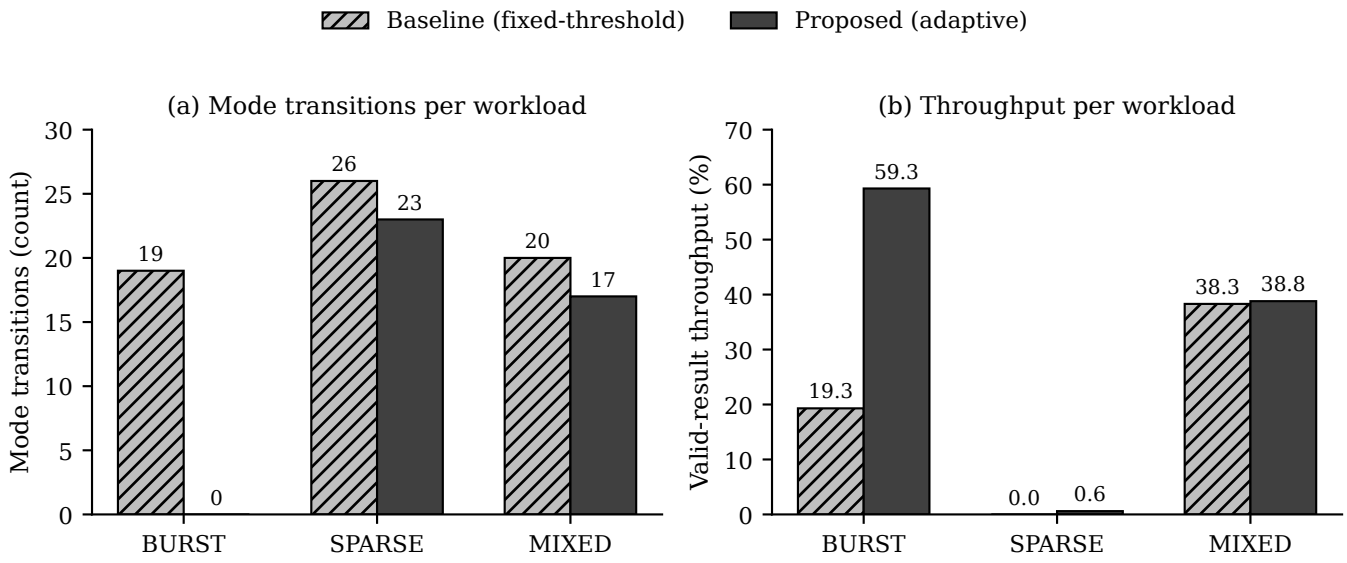

Fig. 9. (a) Mode transitions and (b) valid-result throughput, baseline vs. proposed, across the three evaluated workloads.

TABLE II
PER-MODE CYCLE DISTRIBUTION ACROSS WORKLOADS (BASELINE VS. PROPOSED)

	BURST		**SPARSE**		**MIXED**	
Power State	Base.	Prop.	Base.	Prop.	Base.	Prop.
Cycles ON	78	135	8	16	92	104
Cycles Light Sleep	57	0	10	11	10	9
Cycles Deep Sleep	0	0	155	146	81	70
Total cycles	135	135	173	173	183	183

TABLE III
FUNCTIONAL PERFORMANCE METRICS ACROSS WORKLOADS (BASELINE VS. PROPOSED)

	BURST		**SPARSE**		**MIXED**	
Metric	Base.	Prop.	Base.	Prop.	Base.	Prop.
Mode transitions	19	**0**	26	23	20	17
Wake-up events	9	**0**	8	7	7	6
Leakage saving (%)	8.8	0.7	33.4	**31.6**	17.0	14.7
Throughput (%)	19.3	**59.3**	0.0	0.6	38.3	38.8
Output mismatches	0	0	0	0	0	0

TABLE IV
CELL AREA SUMMARY (DC SYNTHESIS AND POST-ROUTE)

Design point	Baseline (μm^2)	Proposed (μm^2)	Overhead
DC synthesis, LP (5 ns)	350.98	422.51	+20.4%
DC synthesis, HS (2 ns)	351.25	442.76	+26.1%
Post-route (LP corner)	361.50	410.61	+13.6%
Post-route instances	1,014	1,118	+10.3%

TABLE V
TIMING CLOSURE SUMMARY (DC SYNTHESIS AND POST-ROUTE)

	Baseline		Proposed	
Corner	WNS (ns)	TNS (ns)	WNS (ns)	TNS (ns)
DC, LP (5 ns)	+2.44	0.00	+2.51	0.00
DC, HS (2 ns)	+0.01	0.00	+0.01	0.00
Post-route, LP	+3.25	0.00	+3.25	0.00

WNS: worst negative slack (positive = timing met).
TNS: total negative slack (0.00 = no violations).

leakage savings – 31.6% under sparse, 14.7% under mixed – are the operational benefit, measured separately via the cycle-weighted simulation model. The cell placement across the $30\times30\,\mu$m^2 floorplan is shown in Fig. 8; the fully routed layout in Fig. 7.

Figs. 11 and 12 provide magnified detail of the post-route database underlying Fig. 7 and Fig. 8: the former resolves individual net names and routing on a subset of the `u_alu` datapath, and the latter isolates standard-cell instance bound-

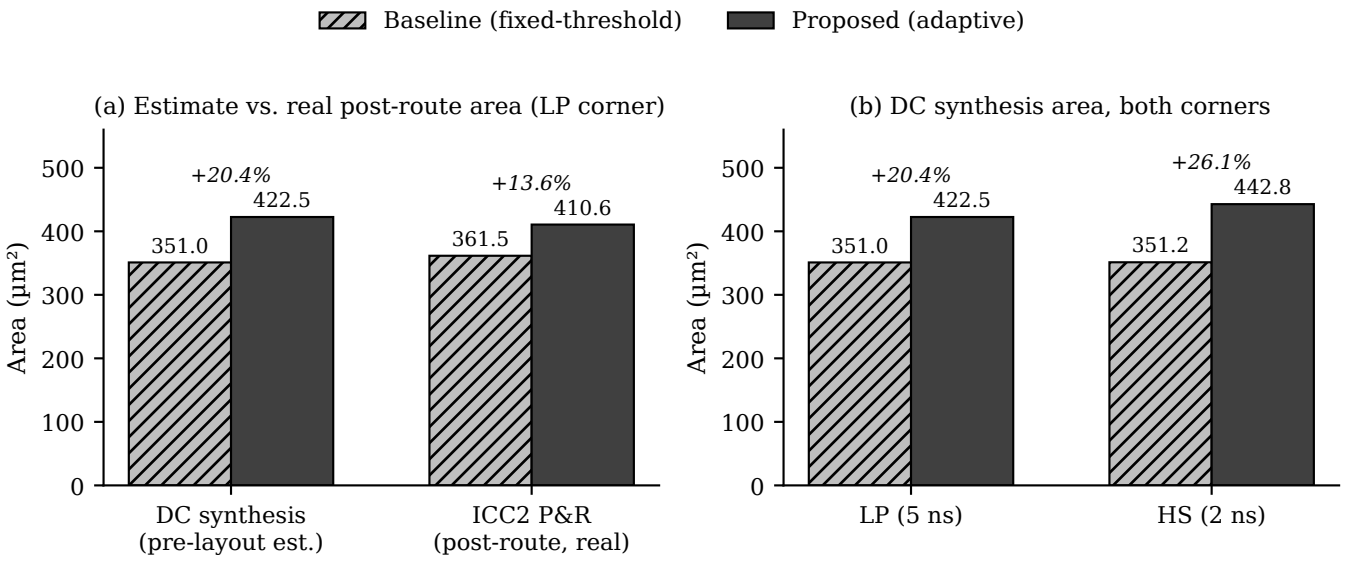

Fig. 10. (a) Pre-layout synthesis estimate vs. real post-route area at the low-power corner, and (b) DC synthesis area at both corners, baseline vs. proposed.

TABLE VI
POWER SUMMARY – DC SYNTHESIS, LP CORNER (5 NS)

Power metric	Baseline	Proposed	Overhead
Cell internal power (μW)	7.09	13.54	+91.0%
Net switching power (μW)	4.71	5.64	+19.7%
Total dynamic power (μW)	11.80	19.18	+62.5%
Cell leakage power (nW)	627.2	991.3	+58.0%

Dynamic power at LP corner; leakage reflects always-on controller cost, not runtime savings during sleep.

TABLE VII
POST-ROUTE STATIC LEAKAGE (LP CORNER)

Metric	Baseline	Proposed	Overhead
Static leakage (pW)	115,164	138,428	+20.2%
Runtime saving, SPARSE (%)	33.4	31.6	−1.8 pp
Runtime saving, MIXED (%)	17.0	14.7	−2.3 pp
Runtime saving, BURST (%)	8.8	0.7	−8.1 pp

Static leakage: full-power cost of controller logic.
Runtime saving: cycle-weighted leakage reduction during actual workload execution (simulation measurement).

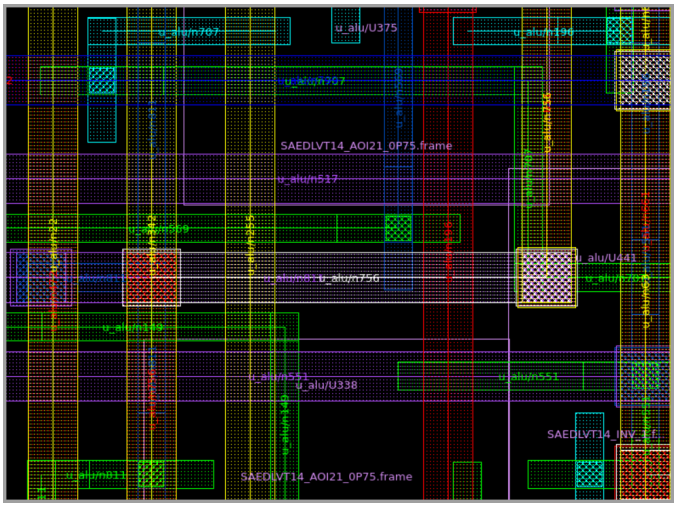

Fig. 11. Zoomed-in post-route view of the `u_alu` region showing individual routed net labels (e.g., `u_alu/n707`, `u_alu/n517`) and standard-cell boundaries within a SAED14LVT `AOI21_0P75` frame, illustrating the interconnect density resolved by the Fusion Compiler router.

aries and identifiers, confirming that the 1,118 placed instances are non-overlapping and legally aligned to the SAED 14 nm LVT placement grid.

VI. DISCUSSION

The central result of this work is the throughput preservation under bursty traffic shown in Section V-A: a 3.1× throughput improvement over a representative fixed-threshold baseline, at zero mode transitions versus nineteen. This directly addresses the thrashing failure mode identified in Section II and confirms the central novelty claim relative to [3].

This benefit is not free. Real post-route synthesis shows a 13.6% area overhead for the additional comparator and threshold-widening logic – below the originally targeted 10% "lightweight" threshold, but a substantial improvement over the 20–26% suggested by pre-layout estimation alone. The appropriate framing for this result is a bounded, quantified trade-off rather than a cost-free improvement: comparator-based adaptive control achieves a meaningful behavioral improvement at a modest, real silicon cost, without resorting to the prediction-hardware overhead of learned approaches such as [5]. The 104 additional standard cells required for the adaptive predictor logic represent a minimal hardware investment compared to the ML inference hardware required by [5].

Compared to prior work: versus [3], this work adds the activity-rate second condition and eliminates thrashing under

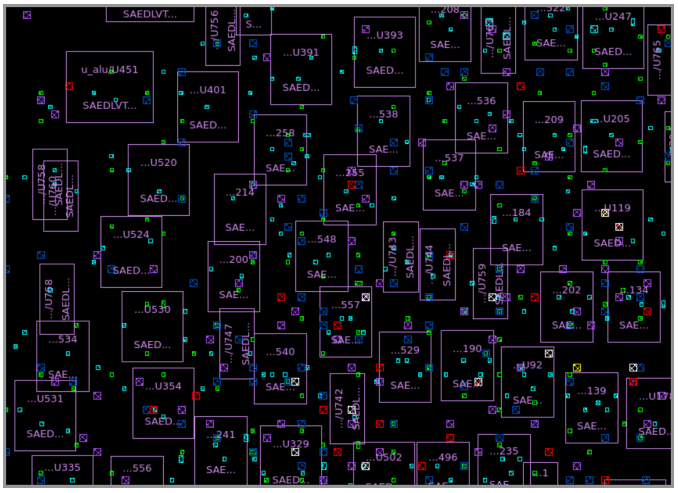

Fig. 12. Zoomed-in cell-level view with instance labels (e.g., `U451`, `U401`, `U520`) overlaid on their corresponding SAED14LVT standard-cell footprints, illustrating instance-level placement density within the $30\times30\,\mu m^2$ core.

burst workloads. Versus [5], this work achieves comparable qualitative adaptivity – suppressing sleep during bursty periods, entering sleep during genuinely idle periods – with no inference hardware, no learned models, and no per-cycle software classification. Versus [4], this work's contribution is orthogonal: Chang et al.'s two-phase wake-up mechanism is adopted unchanged in both baseline and proposed designs as a shared constant.

Limitations: This work's energy comparisons use a derived metric – transitions weighted by maximum wake-up latency – rather than a first-principles joule measurement; a SPICE-level or annotated-switching-activity power simulation would strengthen this claim further. Post-route results are reported at a single LP corner; multi-corner PVT analysis was outside this project's scope. The synthetic workloads, while designed to specifically probe the threshold-crossing behavior under test, are not drawn from a real application trace. DRC verification was limited by the available PDK configuration on the server environment; the 7,376 spacing violations reported by the router reflect missing DRC deck rules rather than functional errors.

VII. Conclusion

This work presented a comparator-based, dual-condition adaptive hysteresis controller for multi-level power gating, evaluated against a fair, structurally matched fixed-threshold baseline at both RTL-simulation and post-route physical-implementation levels. The proposed design eliminates sleep-state thrashing under bursty traffic, preserving throughput at 59.3% versus 19.3% for the fixed-threshold baseline – a 3.1× improvement at zero mode transitions versus nineteen. Under genuinely sparse idle conditions, leakage savings converge to within 1.8 percentage points (31.6% vs. 33.4%), confirming the design does not sacrifice savings when idle conditions are unambiguous. The real post-route area overhead is 13.6%, bounded and materially lower than the pre-layout synthesis estimate of 20–26%. Timing closure is fully met at both LP (5 ns) and HS (2 ns) corners. The complete RTL-to-GDSII implementation was validated through Synopsys VCS functional simulation, DC Ultra synthesis, and Fusion Compiler place-and-route, with 0 open nets and a post-route worst-case slack of 3.25 ns.

Future work includes extending the activity-rate gate to additional functional-block types beyond the ALU, multi-corner PVT physical verification, SPICE-level energy measurement to replace the derived transition-cost proxy metric, and validation against real application traces rather than synthetic workloads.

Code Availability

The RTL, testbench, UPF power intent, and synthesis/place-and-route scripts used in this work are available at https://github.com/JHT127/hysteresis-adaptive-power-gating.

References

[1] G. A. Shiva and M. Patil, "Dynamic voltage scaling for low-power VLSI design: A review of power optimization techniques and memory efficiency enhancements," *International Journal of Environmental Sciences*, vol. 11, no. 23s, 2025.

[2] A. Sahu, S. Khan, S. Nemade, and D. Jain, "Design and optimization of low-power VLSI circuits for high-performance computing," *International Journal of Engineering and Computer Science*, vol. 14, no. 2, pp. 26 804–26 813, 2025.

[3] K. Agarwal, H. Deogun, D. Sylvester, and K. Nowka, "Power gating with multiple sleep modes," in *Proceedings of the 7th International Symposium on Quality Electronic Design (ISQED)*, 2006, pp. 633–637.

[4] C.-Y. Chang, P.-C. Tso, C.-H. Huang, and P.-H. Yang, "A fast wake-up power gating technique with inducing a balanced rush current," in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, 2012, pp. 3086–3089.

[5] M. Karthikeyan, P. Aravinth, and A. J. Heiner, "Adaptive power management techniques in multi core VLSI systems for enhanced energy efficiency," in *Proceedings of the 3rd International Conference on Futuristic Technology (INCOFT)*, vol. 1, 2025, pp. 692–700.

[6] Z. Tong, J. Cai, J. Mei, K. Li, and K. Li, "Computation offloading for energy efficiency maximization of sustainable energy supply network in IIoT," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 2, pp. 128–141, 2024.